

Lab 14: Homework Assignment

Katta Phalgun

(CS22B059)

I did Q1 & Q2. Not attempted Q3.

→ This problem is similar to workflow of IITM.
Hence, I know how to implement it.

I maintained 2 classes, ^{An student* vector}
^{An Course* vector}
Student Course

ID
name
year
no. of Comp courses
Completed Courses
Current Courses
map (Comp. Course)

ID
name
credit
Capacity
slot
no. of Courses
pre-requisites
student list
waitlist

So, for add student,

if student already, return

else add him to the vector

Similarly, for Course,

if pre-requisites doesn't exist, return

if Course already exist, return

If cyclic-dependencies, don't add it.

for Enroll, all these conditions are checked.

→ Student Exist

→ Course Exist

→ If student already Enrolled

→ No Slot clash

→ Course has Capacity.

Then, the Course is added in the Student's Curr Course list.
The student is added to Course's Student list.

Present $\rightarrow$ means present the Course's Student list
else print Invalid if not present.

Cyclic-dependence:

In a for loop, check all the prerequisites.
for each prerequisite, inspect all its prerequisites in a
for loop, if it is the given Course anywhere, reject it.

For Q1, I just wrote naively, by one while loop,
with if add student
else if add Course
Else if Enroll
Else if print
else //

For Q2, I maintained every activity as functions
passing parameters as pointers, not to Create Copies.

$\rightarrow$ Also for drop;

check student Exist.
check Course Exist.

Remove Student from Course's Student list.

Remove Course from student's Course list.

check waiting list of Course & add
eligible student if present.

→ In enroll, if student is not eligible;
he is added to waiting list for slot clash
& Capacity full issue.

Time - Complexity

→ It's very better as I used maps for fast access.